Detailed Explanation of Model Architecture

Math 199 Crypto Pair-Spreads Project

Jack Lutz, Sammy Adham, and Frank Kronewitter

May 27, 2026

Draft note. This document explains the architectures implemented in the current repository. It is a technical draft for project documentation, not a final methods section. If model code changes, this file should be checked against `src/modeling.py`, `src/modeling_big_transformer.py`, `src/kalman_hedge.py`, and `src/hmm_filter.py`.

1 Prediction Task

Each model predicts a three-class label for a pair spread over a 24-hour horizon. Let s_t be the current spread and let s_{t+24} be the future spread. The class label is based on whether the absolute spread moves closer to zero, stays approximately unchanged, or moves farther away:

$$y_t = \begin{cases} 0, & |s_{t+24}| - |s_t| < -\tau \quad \text{(revert)}, \\ 1, & ||s_{t+24}| - |s_t|| \le \tau \quad \text{(persist)}, \\ 2, & |s_{t+24}| - |s_t| > \tau \quad \text{(diverge)}. \end{cases}$$

The fixed threshold is $\tau = 0.001$ in log-spread units unless the per-pair label mode is enabled, in which case τ is estimated from the pair's training-period label volatility.

2 Inputs

The sequential models receive a 168-hour window:

$$X_t \in \mathbb{R}^{168\times 16}.$$

The 16 per-hour feature channels are:

spread,	spread_z_168h,	spread_diff_1h,	spread_diff_4h,
spread_diff_24h,	spread_vol_24h,	spread_vol_168h,	volume_ratio,
volume_z_a_168h,	volume_z_b_168h,	pair_volume_sum,	pair_volume_gmean,
btc_return_24h,	rolling_corr_24h,	realized_vol_a_24h,	realized_vol_b_24h.

The tabular model receives summary statistics computed from the same 168-hour window. These include latest, mean, minimum, and maximum spread z-score; spread slope; latest and mean volume ratio; BTC 24-hour return; rolling correlation; realized volatilities; time since zero crossing; and pair volume summaries.

For learned models, features are standardized using training-window means and standard deviations only:

$$\tilde{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j}.$$

The same training statistics are then applied to the test window.

3 Model Summary

Table 1: Implemented model families.

Model	Input	Trainable?	Output
Persist baseline	none	no	class 1
Majority baseline	labels only	no	training mode
Random stratified	labels only	no	sampled class
Z-score rule	latest spread z	no	class 0 or 1
XGBoost / HGB	14 tabular features	yes	3-class probability
LSTM	168×16 sequence	yes	3 logits
Small transformer	168×16 sequence	yes	3 logits
Big transformer	168×16 sequence	yes	3 logits
Kalman hedge	log-price pair	yes	dynamic residuals
Gaussian HMM	3 spread features	yes	regime state

4 Non-Learned Baselines

4.1 Persist-Class Baseline

The persist baseline always predicts class 1:

$$\hat{y}_t = 1.$$

In the trading convention, class 1 means no new spread trade. This baseline is important because the class distribution can be imbalanced; any learned model should improve on a no-action predictor.

4.2 Majority-Class Baseline

The majority baseline predicts the most common class in the training window:

$$\hat{y}_t = \text{mode}\{y_i : i \in \mathcal{T}_{\text{train}}\}.$$

It uses no features. It tests whether a model is doing more than exploiting the unconditional class distribution.

4.3 Random-Stratified Baseline

The random baseline samples predictions from the empirical training class frequencies:

$$\Pr(\hat{y}_t = k) = \frac{\#\{i \in \mathcal{T}_{\text{train}} : y_i = k\}}{\#\mathcal{T}_{\text{train}}}, \qquad k \in \{0, 1, 2\}.$$

This gives a stochastic null model with the same marginal class distribution as the training data.

4.4 Z-Score Rule

The z-score rule is the main classical trading baseline. It predicts reversion when the absolute spread z-score is large:

$$\hat{y}_t = \begin{cases} 0, & |z_t| \geq 1.5, \\ 1, & |z_t| < 1.5. \end{cases}$$

It never predicts class 2 directly. In the current results, this simple rule has high per-trade win rate because it is selective.

5 Tabular Booster

5.1 Feature Representation

The booster uses the 14 tabular summary features from the 168-hour window:

$$x_t \in \mathbb{R}^{14}.$$

These features compress the recent spread path and market context into a single row per sample. The row is standardized with a training-only `StandardScaler` before fitting.

5.2 XGBoost Architecture

When `xgboost` is available, the model is an `XGBClassifier` with:

max depth	4,
number of trees	300,
learning rate	0.05,
subsample	0.9,
column sample by tree	0.9,
objective	`multi:softprob`,
evaluation metric	`mlogloss`.

Each tree partitions the tabular feature space. The ensemble adds trees sequentially to reduce multiclass log loss:

$$F_m(x) = F_{m-1}(x) + \eta f_m(x),$$

where $\eta = 0.05$ is the learning rate and f_m is the m-th tree. The final prediction is the class with the largest predicted probability:

$$\hat{y}_t = \arg \max_{k \in \{0,1,2\}} p_k(x_t).$$

5.3 Fallback Histogram Gradient Booster

If `xgboost` cannot be imported, the repository falls back to `HistGradientBoostingClassifier` with 300 boosting iterations, learning rate 0.05, and 31 maximum leaf nodes. This preserves the same tabular-boosting role even when the external XGBoost dependency is unavailable.

6 LSTM Classifier

6.1 Architecture

The LSTM receives the full standardized sequence:

$$X_t = (x_{t-167}, \ldots, x_t), \qquad x_i \in \mathbb{R}^{16}.$$

The implemented architecture is:

input size	16,
hidden size	64,
number of recurrent layers	2,
dropout between LSTM layers	0.2,
classifier head	$\mathbb{R}^{64} \to \mathbb{R}^3$.

At each time step, an LSTM cell updates a hidden state h_t and cell state c_t. In simplified notation,

$$h_i, c_i = \mathrm{LSTMCell}(x_i, h_{i-1}, c_{i-1}).$$

After processing all 168 hours, the model keeps the final hidden vector from the last layer:

$$h_{\text{last}} \in \mathbb{R}^{64}.$$

The classifier head maps this vector to three logits:

$$\ell_t = W h_{\text{last}} + b, \qquad \ell_t \in \mathbb{R}^3.$$

The predicted class is $\arg\max_k \ell_{t,k}$.

6.2 Training

The LSTM is trained with cross-entropy loss:

$$\mathcal{L} = -\log \frac{\exp(\ell_{t,y_t})}{\sum_{k=0}^{2} \exp(\ell_{t,k})}.$$

The optimizer is Adam with learning rate 10^{-3}, weight decay 10^{-5}, batch size 256, and the configured number of epochs in the walk-forward script. In the headline deep walk-forward run, the script uses 4 epochs per split.

7 Small Transformer Classifier

7.1 Architecture

The small transformer is a compact encoder intended to be roughly comparable in size to the LSTM. Its architecture is:

input projection	$\mathbb{R}^{16} \to \mathbb{R}^{32}$,
CLS token	1×32 learned vector,
encoder layers	2,
attention heads	2,
model dimension	32,
feedforward dimension	64,
dropout	0.2,
classifier head	$\mathbb{R}^{32} \to \mathbb{R}^3$.

Each hourly feature vector is projected into a 32-dimensional embedding:

$$e_i = W_{\text{in}} x_i + b_{\text{in}}.$$

A learned classification token is prepended:

$$Z_0 = [e_{\text{CLS}}, e_{t-167}, \ldots, e_t].$$

The sequence is passed through two transformer encoder blocks. Each block contains multi-head self-attention followed by a feedforward network. For a single attention head,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

The final CLS embedding is used for classification:

$$\ell_t = W_{\text{head}} z_{\text{CLS}}^{(2)} + b_{\text{head}}.$$

7.2 Training

The small transformer uses the same basic training loop as the LSTM: cross-entropy loss, Adam, learning rate 10^{-3}, weight decay 10^{-5}, batch size 256, and the configured walk-forward epoch count. In the current post-audit headline run, this small transformer underperforms the LSTM and booster; the later big-transformer experiment suggests that this is at least partly a capacity and training-budget issue.

8 Big Transformer Classifier

8.1 Architecture

The big transformer is implemented separately to test whether the small transformer result is caused by insufficient capacity. Its architecture is:

input projection	$\mathbb{R}^{16} \to \mathbb{R}^{128}$,
CLS token	1×128 learned vector,
positional encoding	fixed sinusoidal,
encoder layers	4,
attention heads	4,
model dimension	128,
feedforward dimension	512,
activation	GELU,
dropout	0.2,
normalization	pre-norm encoder plus final LayerNorm,
classifier head	$\mathbb{R}^{128} \to \mathbb{R}^3$.

The projected sequence is augmented with a CLS token and sinusoidal positions:

$$Z_0 = [e_{\text{CLS}}, W_{\text{in}} x_{t-167}, \ldots, W_{\text{in}} x_t] + P,$$

where P is a fixed sinusoidal positional encoding of length 169. The final representation is the normalized CLS output after four encoder blocks:

$$\ell_t = W_{\text{head}}\, \text{LayerNorm}\left(z_{\text{CLS}}^{(4)}\right) + b_{\text{head}}.$$

8.2 Training

The big transformer uses a stronger training regime than the small transformer:

optimizer	AdamW,
learning rate	5×10^{-4},
weight decay	10^{-4},
epochs	20,
batch size	128,
gradient clipping	1.0,
scheduler	cosine decay with 1 warmup epoch,
validation split	last 10% of training data,
early stopping	patience 5 on validation loss.

The validation split is time-ordered, not randomly shuffled, so it better matches the forecasting setup.

9 Kalman Dynamic Hedge Model

9.1 Purpose

The Kalman model is not a three-class predictor. It is the dynamic hedge-ratio model used to construct better spreads before feature engineering. It replaces a static OLS hedge ratio with time-varying intercept and slope states.

9.2 State-Space Model

For log prices $y_t = \log(P_{a,t})$ and $x_t = \log(P_{b,t})$, the state is

$$\theta_t = \begin{bmatrix} \alpha_t \\ \beta_t \end{bmatrix}.$$

The transition model is a random walk:

$$\theta_t = \theta_{t-1} + w_t, \qquad w_t \sim \mathcal{N}(0, Q),$$

with diagonal process covariance

$$Q = \begin{bmatrix} q_\alpha & 0 \\ 0 & q_\beta \end{bmatrix}.$$

The observation model is:

$$y_t = \begin{bmatrix} 1 & x_t \end{bmatrix} \theta_t + v_t, \qquad v_t \sim \mathcal{N}(0, R).$$

The Kalman residual is the one-step innovation:

$$\epsilon_t = y_t - \begin{bmatrix} 1 & x_t \end{bmatrix} \hat{\theta}_{t|t-1}.$$

These residuals become the dynamic spread used by the downstream pipeline.

9.3 Parameter Fitting

Initial α_0 and β_0 are seeded from OLS. The noise parameters q_α, q_β, and R are fit on the training slice by minimizing the Kalman negative log likelihood:

$$\mathcal{L} = \sum_t \frac{1}{2}\left[\log(2\pi S_t) + \frac{\epsilon_t^2}{S_t}\right],$$

where S_t is the innovation variance. Fitting uses L-BFGS-B in log parameter space. In out-of-sample evaluation, the fitted parameters and final training state are carried forward; the test window is not refit.

10 Gaussian HMM Regime Filter

10.1 Purpose

The HMM is a regime filter rather than a standalone predictor. It attempts to detect whether a pair is in a mean-reverting regime or a diverging regime, then suppresses trades outside the mean-reverting state.

10.2 Inputs and Hidden States

The HMM uses three spread-derived features:

$$x_t = \begin{bmatrix} \text{spread\_z\_168h}_t \\ \text{spread\_diff\_1h}_t \\ \text{spread\_vol\_24h}_t \end{bmatrix}.$$

The default model has $K = 2$ hidden states. The hidden state follows a Markov chain:

$$z_t \sim \text{Categorical}(A_{z_{t-1},:}),$$

and observations are Gaussian conditional on the state:

$$x_t \mid z_t = k \sim \mathcal{N}(\mu_k, \Sigma_k).$$

The implementation uses full covariance matrices and expectation-maximization through `hmmlearn`.

10.3 Mean-Reverting State Identification

After fitting, the model decodes states with Viterbi. The state with lower standard deviation of spread_diff_1h is labeled "mean-reverting." If there is a tie, the state with lower mean absolute spread z-score is chosen. Predictions from other models are then filtered:

$$\hat{y}_t^{\text{filtered}} = \begin{cases} \hat{y}_t, & z_t = z_{\text{MR}}, \\ 1, & z_t \neq z_{\text{MR}}. \end{cases}$$

Class 1 is the flat/no-trade class.

11 Training and Walk-Forward Evaluation

The main walk-forward driver trains models independently on each split:

$$90 \text{ days train} \rightarrow 30 \text{ days test},$$

then advances by 30 days. The long Kalman pipeline produces 27 splits. Each split trains from scratch using only training-window samples and evaluates on the held-out test window.

For deep models, the sequence tensor is indexed by sample id so every model uses the same examples. For the tabular model, the same sample ids are mapped to window-level summary features. This keeps model comparisons aligned across the same pair-time observations.

12 Prediction-to-Trade Convention

Model outputs are class labels, not direct portfolio weights. The simple model comparison metric translates classes into spread-direction signals:

$$\text{signal}_t = \begin{cases} -\operatorname{sign}(s_t), & \hat{y}_t = 0 \quad \text{(revert)}, \\ 0, & \hat{y}_t = 1 \quad \text{(persist)}, \\ \operatorname{sign}(s_t), & \hat{y}_t = 2 \quad \text{(diverge)}. \end{cases}$$

This convention is used for comparing predictive signal quality. The separate portfolio backtester adds position state, notional sizing, and transaction costs.

13 Interpretation

The architecture stack is deliberately layered. The Kalman model defines a dynamic residual spread. The feature builder converts that spread into backward-looking sequence and tabular representations. Baseline, z-score, booster, LSTM, and transformer models then predict whether the spread should revert, persist, or diverge. Finally, the HMM can be applied as an optional regime filter, and the backtester converts predictions into positions.

The current project results suggest that the strongest statistical advance is the Kalman dynamic spread construction. The learned prediction models improve some signal-quality metrics, but much of their useful information appears to come from the latest spread z-score. This is why the architecture documentation should be read together with the cost and selection-bias results in the main paper draft.